

Lista de Exercícios de Fixação

1. Explique as vantagens da documentação automatizada de código e como ela pode beneficiar uma equipe de desenvolvimento.

- **Redução de erros:** a documentação reflete o código atual, evitando desatualizações.
- **Economia de tempo:** os desenvolvedores não precisam escrever e manter documentação manual em paralelo ao código.
- **Consistência:** como a documentação é gerada a partir do próprio código-fonte, ela sempre acompanha as mudanças.

Para uma equipe de desenvolvimento, isso significa menos discrepância entre o que está documentado e o que está implementado, além de liberar tempo para funcionalidades em vez de manutenção manual de documentos.

2. Crie um pequeno projeto que utilize o MKDocs integrado ao Swagger para documentar uma API de um sistema fictício de gerenciamento de usuários.

Estrutura de pastas:

```
meu-projeto/  
  mkdocs.yml  
  docs/  
    index.md  
    api/  
      openapi.yaml  
      swagger-ui/  
        index.html
```

Arquivo docs/api/openapi.yaml com os endpoints principais de usuários:

```
openapi: 3.0.0  
info:  
  title: API de Usuários  
  version: 1.0.0  
paths:  
  /usuarios:  
    get:  
      summary: Lista todos os usuários  
      responses:  
        '200':  
          description: Lista retornada com sucesso
```

O mkdocs.yml foi atualizado para incluir o Swagger UI no menu.

3. Usando a especificação OpenAPI, documente uma API com pelo menos três endpoints: um para listar, um para criar e um para atualizar recursos. Gere e visualize a documentação com o MKDocs.

```
openapi: 3.0.0  
info:  
  title: API de Tarefas  
  version: 1.0.0  
paths:  
  /tarefas:
```

```

get:
  summary: Lista todas as tarefas
  responses:
    '200':
      description: Lista retornada com sucesso
post:
  summary: Cria uma nova tarefa
  responses:
    '201':
      description: Tarefa criada com sucesso
/tarefas/{id}:
  put:
    summary: Atualiza uma tarefa existente
    responses:
      '200':
        description: Tarefa atualizada com sucesso

```

Com o arquivo salvo em docs/api/openapi.yaml e o Swagger UI configurado, mkdocs serve e exibe a documentação interativa, permitindo testar os três endpoints diretamente pelo navegador.

4. Pesquise sobre outra ferramenta de documentação automatizada (como Doxygen ou Sphinx) e explique como ela pode ser integrada ao MKDocs.

O Doxygen gera documentação a partir de comentários no código-fonte (em C, C++, Java, entre outras linguagens), criando páginas HTML com a estrutura de classes, funções e suas descrições.

Para integrar ao MKDocs, o fluxo comum é: gerar a documentação HTML com Doxygen, salvar o resultado dentro da pasta docs/ do projeto MKDocs (por exemplo, docs/doxygen/) e adicionar um link para esse HTML na navegação do mkdocs.yml, da mesma forma como foi feito com o Swagger UI.

5. Modifique o exemplo fornecido da API de produtos, adicionando um endpoint para excluir um produto, e atualize a documentação automaticamente com o Swagger.

```

paths:
  /produtos/{id}:
    delete:
      tags:
        - Produtos
      summary: Exclui um produto pelo ID
      responses:
        '204':
          description: Produto excluído com sucesso
        '404':
          description: Produto não encontrado

```

Após salvar o openapi.yaml atualizado, basta rodar mkdocs build novamente; o Swagger UI recarrega a especificação automaticamente e passa a exibir o novo endpoint DELETE /produtos/{id} na interface interativa.